

RICORSIONE:

forma di utilizzo di Astrazione del codice per poter Iterare in modo Alternativo

↪ Approcci diversi ma INTERSCAMBIABILI

Se RICORSIVA ⇒ Se Richiama se Stessa. (può NON Terminare)

Esempio:

```
fattoriale(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fatt(n-1);  
}
```

Informatica

Parametro DECRESCERE e mi Garantisce la TERMINAZIONE

Assomiglia All' INDUZIONE:

$$\begin{cases} \text{fatt}(1) = \text{fatt}(0) = 1 \\ \text{fatt}(n) = n \cdot \text{fatt}(n-1) \end{cases}$$

Matematica

Così facendo NON Abbiamo detto NESSUN Vincolo per Mantenere la TURING CALCOLABILITÀ

$\text{fie}(1) = \text{fie}(1)$ NON È INDUZIONE

Ma possiamo Scrivere un PROGRAMMA che si Comporti Così,
Quindi che diverge

Altro Esempio:
$$\begin{cases} \text{foo}(0) = 0 \\ \text{foo}(n) = \text{foo}(n+1) \end{cases}$$

Non Tende Verso La Base, Parametro Cresce $\Rightarrow$ non è INDUTTIVA
anche se ESISTE un Programma che si Comporta Così

PERCHÈ un LINGUAGGIO PERMETTA LA RICORSIONE ABBIAMO BISOGNO:

/// Procedure

/// Gestione Dinamica della MEMORIA (no alloc. Statica)

RICORSIONE IN CODA:

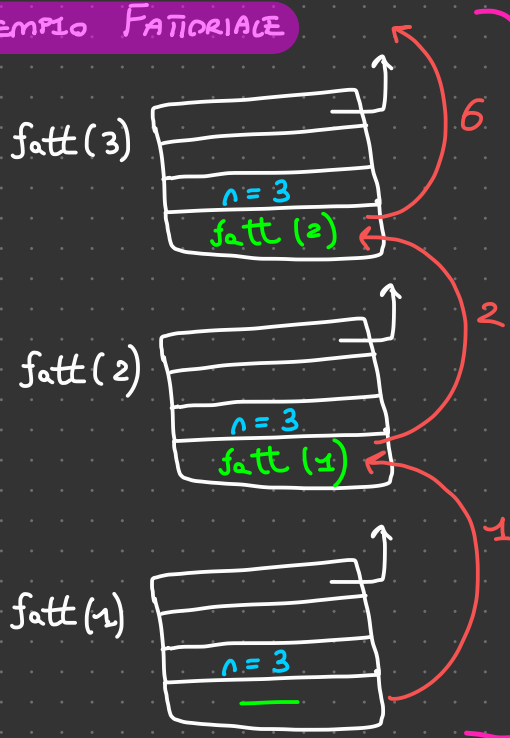
OTTIMIZZA l'uso della MEMORIA della RICORSIONE

Una CHIAMATA dentro f ad una PROCEDURA g si dice in CODA se f restituisce il Valore RITORNATO da g senza ulteriori COMPUTAZIONI

Una procedura si dice ricorsiva in CODA se la chiamata ricorsiva è in CODA.

fattoriale Viene MANIPOLATO (moltiplicato) $\Rightarrow$ non in CODA.

ESEMPIO FATTORIALE



Se NON è di CODA NON POSSO SOVRASCRIVERE il Record di Attivazione Precedente perché devo MEMORIZZARE il valore Per poi Elaborarlo.

FATTORIALE IN CODA:

```
int fattRC(int n, int res) {  
    if  $n \leq 1$   
        return RES  
    else  
        return fattRC( $n-1$ ,  $\text{res} \cdot n$ )  
}
```

Resultato parziale = Caso Base All'inizio

Valore che ho costruito Nelle Chiamate precedenti prima del caso BASE

```
int fatt(int n) {  
    fattRC( $n$ , 1);  
}
```

Solitamente ho Almeno 1 Parametro in Più Per la RICORSIONE IN CODA

$n = 3$

$\text{f_att}(3)$

$\text{f_attRC}(3, 1)$

$\text{f_attRC}(2, 3)$

$\text{f_attRC}(1, 6)$

Sono al CASO BASE e
return di RESULT che ho
costituito nel PARAMETRO

FIBONACCI:

$\text{Fib}(0) = 1$

$\text{Fib}(1) = 1$

$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

```
int Fib(int n) {
```

```
    if  $n \leq 1$ 
```

```
        return n;
```

```
    else
```

```
        return  $\text{Fib}(n-1) + \text{Fib}(n-2)$ ;
```

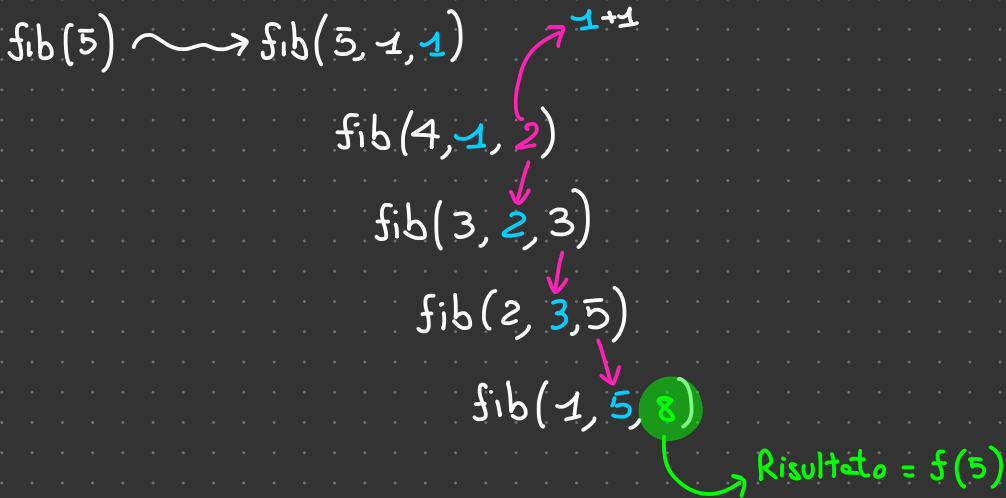
→ NON è in coda visto che c'è una
MANIPOLAZIONE

mi servono i 2 Passi precedenti

```
int FibRC(int n, int res1, int res2) {  
    if n <= 1  
        return res2 → Quello da RESTITUIRE / CORRETTO  
    else  
        return Fib(n-1, res, res1+res2)
```

```
int fib(n) {  
    return fibRC(n, 1, 1)
```

ESEMPIO:



TIPOLOGIE DI ESERCIZI:

/// domanda di Teoria Sull'Implementazione del linguaggio

/// Interpreti } def. Intuitiva & Formale
/// Compilatori }

/// Esercizi

/// Induzione Strutturale (no Matematica) (DIMOSTRARE)

/// Scoping → Catena Statica + CRT (DA COSTRUIRE)
→ Codice da Completare

/// Binding (F.Z. COME PARAMETRO)

/// Passaggio Parametri (VALORE / RIFERIMENTO)

/// Ricorsione (da NORMALE a CODA)

/// Di Semantica (ASSEGNAME~~N~~TO/DICHIARAZIONE)

COMPLETAMENTO CODICE:

Dato un **Pezzo di CODICE**:

```
{int i;
```

⊛

```
for(int i=0; i<1; i++){
```

```
    int x;
```

```
    x = fun();
```

```
}
```

Fornire il codice da inserire in ⊛ t.c. il linguaggio ha SCOPING Statico le 2 chiamate alla funzione restituiscono lo stesso Valore.

Con SCOPING DINAMICO invece devono ESSERE DIVERSE e descrivere la Scelta.

Codice deve Essere Esegribile Quindi deve ESSERE definito tutto ciò che si Usa.

Bisogna definire una funzione fun()

Perchè il **COMPORTAMENTO** sia **POTENZIALMENTE diverso** per via delle REGOLE di SCOPING (AMBIENTE NON LOCALE) è Necessario che ci Sia:

1. **Procedura** con **Ambiente non Locale**

2. **Riferimento** non locale deve **∃** sia **Nel Contesto di**

definizione che di Chiamata

i è un Buon Candidato per Essere un AMBIENTE non locale per fun
x NON È INIZIALIZZATA Nell' AMBIENTE DI CHIAMATA e NON può
ESSERE UTILIZZATA

Se ci fossimo 2 (*) allora avremmo potuto inizializzare.

Codice All' Interno di (*)

$i = 1;$ → Per Rendere codice Eseguitibile

```
int fun() {  
    return i;  
}
```

→ Ad Ogni ITERAZIONE i cambia per Ogni CHIAMATA.

ESECUZIONE SCOPING STATICO

Chiamata Restituisce la i Globale = 1 che NON Viene Modificata

ESECUZIONE SCOPING DINAMICO

Le 2 Chiamate di $fun()$ fanno riferimento alla i del ciclo
for che CAMBIA:

① return 0;

② return 1;

ALTRO ESEMPIO:

```
{ int x
```

⊗

```
void exec(){
```

```
  for(int J=2; J≤3; J++){
```

```
    int y = 1;
```

```
    x = fun();
```

```
  }
```

```
}
```

```
exec();
```

```
}
```

Ambiente di Chiamata non
MODIFICABILE

→ È Sempre UGUALE

⊗

```
Int J = 99;
```

```
fun(){
```

```
  return J;
```

```
}
```

ESECUZIONE SCOPING STATICO

J=99 viene prese Sempre IGNORANDO la Variabile J del
Ciclo che Quella VARIA

ESECUZIONE SCOPING DINAMICO

Le ITERAZIONI del ciclo Portano Risultati diversi:

- ① return 2;
 - ② Return 3;
- } Se Avessi USATO y Allora Avrei Avuto
Sempre RETURN 1;

ALTRO ESERCIZIO:

```
int a = 0;
(*)
{while (a < 1)}
  int x;
  (**)
  x = fun();
  a++;
}

(*) int z = -10;
  fun() {
    return z;
  }

(**) int z = 4 + a;
```

ESECUZIONE SCOPING STATICO

Chiamata Restituire -10 perché si riferisce alla z Statica

ESECUZIONE SCOPING DINAMICO

Chiamate:

① RETURN 4;

② RETURN 5;

a NON può ESSERE Usata visto che a Esiste Solo GLOBALE e se la RIDEFINISSI potrei ROMPERE il WHILE

Ma x è una Buona Candidata (con dovute Modifiche nell' Ambiente di Chiamata).

DA FARE PER CASA :

GUARDARE SLIDES